

Brailify Repository Analysis

작성일: 2026-05-22

대상 워크트리: [/Users/junmin/Documents/04_Projects/26_한이음/brailify]

원본 저장소: [https://github.com/dev-five-git/brailify]

이 문서는 Brailify 저장소를 처음 맡은 개발자가 전체 구조를 이해할 수 있도록 작성한 코드리뷰형 분석 문서다. 핵심 관점은 다음과 같다.

- 이 프로젝트의 진짜 중심은 Rust crate (libs/brailify)다.
- Node, Python, .NET, CLI, 웹사이트, 모바일 앱은 모두 Rust 코어를 감싸는 소비자다.
- 점역 로직은 단순 문자 매핑이 아니라 (토큰화 -> 토큰 규칙 -> 문자 규칙 -> 출력) 흐름으로 처리된다.
- 테스트케이스는 (docs/2024 개정 한국 점자 규정.pdf)를 근거로 삼는 사실상의 제품 명세다.

1. 저장소 전체 구조

루트 디렉터리의 주요 구성은 다음과 같다.

```
.
├── libs/brailify/      # Rust 핵심 점역 엔진
├── packages/node/      # Node.js / WebAssembly 바인딩
├── packages/python/    # Python / maturin / pyo3 바인딩
├── packages/dotnet/    # .NET 바인딩과 Rust C ABI 네이티브 라이브러리
├── apps/landing/       # Next.js 공식 웹사이트
├── apps/mobile/        # Vite + React + Tauri 모바일/데스크톱 앱
├── test_cases/         # 한국어/수학 점자 규정 기반 테스트케이스
├── docs/               # 2024 개정 한국 점자 규정 PDF
├── braillove-case-collector/ # 내부 표기 -> expected/unicode 변환 보조 도구
├── scripts/            # 외부 점역기 비교 데이터 수집 스크립트
├── __tests__/          # Bun 기반 Node 패키지 테스트
├── .github/workflows/  # CI, 배포, 패키지 publish workflow
├── Cargo.toml          # Rust workspace
├── package.json        # Bun workspace와 통합 스크립트
├── pyproject.toml      # uv Python workspace
└── rule_map.json       # 테스트케이스 조항 메타데이터
```

현재 로컬 워크트리에서는 (package.json)이 수정되어 있고 (apps/mobile/)은 비추적 파일로 존재한다. 따라서 원본 저장소 분석과 현재 작업물 분석을 구분해야 한다.

2. 빌드 시스템과 워크스페이스

2.1 Rust workspace

루트 (Cargo.toml)은 다음처럼 모든 (libs/*), (packages/*)를 workspace member로 묶는다.

```
[workspace]
members = ["libs/*", "packages/*"]
resolver = "3"
```

중요한 점은 [packages/node], [packages/python], [packages/dotnet]도 Rust crate라는 것이다. 각각은 언어별 바인딩 crate이고, 실제 점역은 [libs/brailify]를 dependency로 호출한다.

2.2 Bun workspace

루트 [package.json]은 Node 패키지와 앱을 묶는다.

```
"workspaces": [  
  "packages/node",  
  "apps/landing",  
  "apps/mobile"  
]
```

주요 스크립트는 다음 성격이다.

- [bun run test]: Rust coverage, Bun test, Python pytest까지 묶은 통합 테스트
- [bun run build]: Rust release build, Node WASM build, Python wheel build
- [bun run dev]: landing 앱 개발 서버 실행
- [bun run build:landing]: 코어/테스트 일부와 landing build 실행

개발자가 조심할 점은 [preinstall]에서 [uv sync], [cargo install wasm-pack], [pip install maturin] 같은 설치 작업을 한다는 것이다. CI나 새 환경에서는 이 단계가 오래 걸릴 수 있다.

2.3 Python workspace

루트 [pyproject.toml]은 [packages/python], [py-test]를 uv workspace member로 둔다. Python 패키지 자체는 [packages/python/pyproject.toml]에서 maturin으로 빌드된다.

3. 핵심 Rust 엔진: [libs/brailify]

[libs/brailify]는 이 프로젝트의 심장이다.

```
libs/brailify/  
├── Cargo.toml  
├── build.rs  
└── src/  
    ├── lib.rs  
    ├── encoder.rs  
    ├── cli.rs  
    ├── main.rs  
    ├── char_struct.rs  
    ├── korean_char.rs  
    ├── char_shortcut.rs  
    ├── word_shortcut.rs  
    ├── english.rs  
    ├── english_logic.rs  
    ├── number.rs  
    ├── symbol_shortcut.rs  
    ├── math_symbol_shortcut.rs  
    ├── fraction.rs  
    ├── unicode.rs  
    └── split.rs
```

```

├── korean_part.rs
├── jauem/
├── moeum/
└── rules/

```

3.1 공개 API

외부 사용자가 직접 쓰는 함수는 [src/lib.rs]에 있다.

```

pub fn encode(text: &str) -> Result<Vec<u8>, String>
pub fn encode_with_options(text: &str, options: &EncodeOptions) -> Result<Vec<u8>, String>
pub fn encode_with_formatting(text: &str, spans: &[FormattingSpan]) -> Result<Vec<u8>, String>
pub fn encode_to_unicode(text: &str) -> Result<String, String>
pub fn encode_to_unicode_with_formatting(text: &str, spans: &[FormattingSpan]) -> Result<String, String>
pub fn encode_to_braille_font(text: &str) -> Result<String, String>

```

[encode]의 반환값은 점자 셀 인덱스 [Vec<u8>]이다. 예를 들어 [0]은 공백 점자 셀, [60]은 수표 등으로 해석된다. [encode_to_unicode]는 이 셀 인덱스를 Unicode Braille block 문자로 바꾼다.

[encode_to_braille_font]는 현재 구현상 [encode_to_unicode]와 사실상 동일하게 Unicode Braille 문자를 반환한다. 이름만 보면 별도 폰트용 코드 포인트를 줄 것 같지만, 지금은 [unicode::encode_unicode]를 그대로 사용한다.

3.2 인코딩의 큰 흐름

처리 흐름은 다음 순서다.

```

input text
-> encode / encode_with_options
-> Encoder::new(english_indicator)
-> DocumentIR::parse
-> TokenRuleEngine::apply_all
-> optional formatting token injection
-> emit
-> RuleEngine::apply_phase(CoreEncoding / InterCharacter)
-> Vec<u8>
-> Unicode braille string

```

주니어 개발자가 처음 볼 때 가장 중요한 파일은 다음 순서다.

1. [libs/brailify/src/lib.rs]
2. [libs/brailify/src/encoder.rs]
3. [libs/brailify/src/rules/token.rs]
4. [libs/brailify/src/rules/token_engine.rs]
5. [libs/brailify/src/rules/emit.rs]
6. [libs/brailify/src/rules/traits.rs]
7. [libs/brailify/src/rules/engine.rs]
8. [libs/brailify/src/korean_char.rs]
9. [libs/brailify/src/rules/korean/rule_korean.rs]

이 순서대로 보면 "입구", "중간 표현", "규칙 적용", "문자 인코딩"이 자연스럽게 이어진다.

4. [Encoder] 구조

[encoder.rs]의 [Encoder]는 상태와 두 종류의 규칙 엔진을 가진다.

```
pub struct Encoder {
    pub(crate) is_english: bool,
    triple_big_english: bool,
    english_indicator: bool,
    has_processed_word: bool,
    pub(crate) needs_english_continuation: bool,
    parenthesis_stack: Vec<bool>,
    default_mode: Option<EncodingMode>,
    rule_engine: rules::engine::RuleEngine,
    token_engine: rules::token_engine::TokenRuleEngine,
}
```

중요한 상태는 다음과 같다.

- [english_indicator]: 입력 전체에 한글이 있는지 기반으로 영어 표지 사용 여부를 결정한다.
- [is_english]: 현재 로마자 구간 안에 있는지 나타낸다.
- [triple_big_english]: 대문자 passage 처리 상태다.
- [needs_english_continuation]: 영어 구간을 끊었다가 이어갈 때 continuation marker를 넣기 위한 상태다.
- [parenthesis_stack]: 영어 문맥 안의 괄호 처리를 추적한다.
- [default_mode]: Korean, English, Math, MiddleKorean 같은 기본 모드 override 용도다.

4.1 RuleEngine 등록 순서

[Encoder::new]에서 한글 규칙을 직접 등록한다. 등록은 크게 다음 그룹이다.

- Preprocessing: 제53항 말줄임표 정규화
- WordShortcut: 제18항 단어 약어
- ModeManagement: 제29항 로마자 표지
- CoreEncoding: 한글, 영어, 숫자, 기호, 중세국어, 수학 기호 등 대부분의 핵심 처리
- InterCharacter: 제11항, 제12항처럼 글자 사이에 붙임표/구분표가 필요한 규칙

이 등록 순서 자체가 "이 엔진이 어떤 우선순위로 점역을 판단하는가"를 보여준다. 기능 추가 시 새 규칙을 아무 데나 넣으면 회귀가 생긴다. 반드시 phase와 priority를 기준으로 넣어야 한다.

4.2 TokenRuleEngine 등록 순서

토큰 규칙은 문자 단위보다 먼저 작동한다. 등록되는 규칙은 다음 성격이다.

- 중세국어 감지
- 중세국어 주석 간격

- 말줄임표 정규화
- LaTeX math merge
- 강조 원문자 처리
- 일반 수학 표현 감지
- LaTeX fraction
- LaTeX math
- inline fraction
- 단어 약어
- 로마 숫자
- 디지털 표기
- 대문자 passage
- 가운데점 spacing
- quote attachment
- asterisk spacing

토큰 규칙은 “문자 하나만 봐서는 모르는 것”을 처리한다. 예를 들어 $\{\frac{3}{4}\}$, $\{1/2\}$, $\{WELCOME TO KOREA\}$, $\{IV\}$, $\{example.com/path\}$ 같은 것은 단순 char loop보다 토큰 단계에서 다루는 것이 맞다.

5. DocumentIR와 토큰 모델

`[rules/token.rs]`는 중간 표현을 정의한다.

```
pub enum Token<'a> {
    Word(WordToken<'a>),
    Space(SpaceKind),
    Fraction(FractionToken),
    Mode(ModeEvent),
    PreEncoded(Vec<u8>),
}
```

각 토큰의 의미는 다음과 같다.

- `[Word]`: 일반 텍스트 단어
- `[Space]`: 일반 공백
- `[Fraction]`: 분자/분모/대분수 정보를 가진 분수 토큰
- `[Mode]`: 영어 진입, 영어 continuation, 대문자 단어, 대문자 passage 시작/끝 같은 모드 이벤트
- `[PreEncoded]`: 이미 점자 바이트로 변환된 결과

`[DocumentIR::parse]`는 기본적으로 공백 기준으로 단어를 나눈다. 단, $\{\dots\}$ LaTeX 수식은 내부에 공백이 있어도 하나의 토큰으로 합친다. 이 처리가 없으면 $\{\int f(x) dx\}$ 같은 입력이 단어별로 쪼개져 수식 파서가 실패한다.

`[WordMeta]`는 각 단어에 대한 빠른 힌트다.

- {has_korean}
- {is_all_uppercase}
- {starts_with_ascii}
- {has_ascii_alphabetic}

이 메타는 영어 표지, 대문자 표지, 혼합 단어 처리를 빠르게 결정하는 데 쓰인다.

6. emit 단계

{rules/emit.rs}는 토큰을 최종 점자 бай트로 방출한다.

핵심 함수는 {emit}과 {emit_word}다.

{emit}은 토큰 종류별로 처리한다.

- {Token::Word}: {emit_word}
- {Token::Space}: {0} 방출
- {Token::Mode}: 영어/대문자 marker 방출
- {Token::Fraction}: {fraction::encode_fraction} 또는 {encode_mixed_fraction}
- {Token::PreEncoded}: 그대로 결과에 붙임

{emit_word}는 문자 단위 loop를 돈다. 여기서 {CharType::new}로 현재 문자를 분류하고, 영어 모드 진입/종료, 숫자 상태, 대문자 상태, 괄호 상태 등을 갱신한다. 그 후 {RuleEngine}의 {CoreEncoding} phase를 적용하고, 한글 음절이면 필요 시 {InterCharacter} phase를 적용한다.

리뷰 관점에서 {emit_word}는 매우 중요한 파일이지만 복잡도가 높은 편이다. 현재는 많은 상태 전이가 한 함수 안에 모여 있다. 기능 수정 시에는 작은 입력을 만들어 {encode_to_unicode} 결과를 먼저 확인하고, 기존 테스트케이스 전체를 돌려야 한다.

7. 문자 분류: {CharType}

{char_struct.rs}의 {CharType}은 엔진이 입력 문자를 어떻게 붙지 결정한다.

```
pub enum CharType {
    Korean(KoreanChar),
    KoreanPart(char),
    English(char),
    Number(char),
    Symbol(char),
    MathSymbol(char),
    Fraction(char),
    CombiningMark,
    Space(char),
}
```

이 분류는 생각보다 중요하다. 어떤 문자가 (Symbol)로 들어가면 일반 기호 규칙을 타고, (MathSymbol)로 들어가면 수학 기호 규칙을 타며, (KoreanPart)로 들어가면 standalone jamo나 중세국어 관련 규칙을 탄다.

특히 이 파일은 넓은 유니코드 범위를 허용한다.

- 현대 한글 음절
- 호환 자모
- 옛한글 자모
- CJK 한자
- IPA 문자
- Greek
- Latin extended
- 전각 문자
- 일반 구두점
- 기하 도형
- 개인 사용 영역

이는 테스트케이스가 단순 현대 한글만이 아니라 중세국어, 외국어, IPA, 수학, 기호를 포함하기 때문이다.

8. 한글 음절 인코딩

8.1 한글 분해

(KoreanChar::new)는 유니코드 한글 음절 공식으로 초성/중성/종성을 분해한다.

```
code - 0xAC00
초성 index = uni / 588
중성 index = (uni % 588) / 28
종성 index = uni % 28
```

이 로직은 표준적인 방식이다. 조합형 한글을 직접 테이블로 들고 있지 않아도 모든 현대 한글 음절을 분해할 수 있다.

8.2 초성/중성/종성 표

초성 표는 {jauem/choseong.rs}, 중성 표는 {moeum/jongsong.rs}, 종성 표는 {jauem/jongseong.rs}에 있다.

초성 (ㅇ)은 초성 위치에서는 점역하지 않는다. 예를 들어 {아}는 초성 (ㅇ)을 생략하고 중성 (ㅏ)만 출력한다. 이 때문에 {encode_korean_char}에는 {if cho0 != 'ㅇ'} 조건이 반복된다.

8.3 약자 우선 처리

[`encode_korean_char`]는 무조건 초성/중성/종성을 각각 붙이지 않는다. 먼저 [`char_shortcut::encode_char_shortcut`]를 시도한다.

예를 들어 다음 글자들은 약자 테이블에 있다.

- [가], [나], [다], [마], [바], [사], [자], [카], [타], [파], [하]
- [것]
- [억], [언], [얼], [연], [열], [영]
- [옥], [온], [옹], [운], [울], [은], [을], [인]
- [성], [정], [청]

이 우선순위가 중요하다. 약자 적용 여부에 따라 결과가 완전히 달라진다.

9. 한글 규정 파일

[`rules/korean/`]은 조항별로 파일이 나뉜다. 예시는 다음과 같다.

- [`rule_1.rs`]: 기본 초성
- [`rule_2.rs`]: 된소리 초성
- [`rule_3.rs`]: 기본 종성
- [`rule_8.rs`]: 낱자 자모 표기
- [`rule_11.rs`]: [예] 앞 구분
- [`rule_12.rs`]: [애] 앞 구분
- [`rule_13.rs`]: 음절 약자
- [`rule_14.rs`]: 모음 시작 음절 앞 약자 제한
- [`rule_16.rs`]: 예외 음절 분해
- [`rule_18.rs`]: 단어 약어
- [`rule_19`]-[`rule_27`]: 중세국어 관련 처리
- [`rule_28.rs`]: 영어 글자 인코딩
- [`rule_29.rs`]: 로마자 표지
- [`rule_31.rs`]: 그리스 문자
- [`rule_40.rs`]: 수표
- [`rule_41.rs`]: 숫자 중 십표
- [`rule_44.rs`]: 숫자와 혼동 가능한 한글 초성 사이 띄어쓰기
- [`rule_49.rs`]: 문장부호
- [`rule_53.rs`]: 말줄임표 정규화
- [`rule_56.rs`]: 강조 결합 문자
- [`rule_57.rs`]: 반복 기호 묶음

- [rule_58.rs]: 빈칸 기호
- [rule_60.rs]: 별표 spacing
- [rule_61.rs]: 숫자 앞 apostrophe
- [rule_64.rs]: 원문자/괄호문자
- [rule_65.rs]: 통화 기호
- [rule_66.rs]: 점자 셀 literal
- [rule_67.rs]: 본문 중 점자 셀 언급
- [rule_68.rs]: 위첨자/아래첨자와 일부 단위 기호
- [rule_69.rs]: 단위 기호
- [rule_70.rs]: 화살표
- [rule_71.rs]: 정보/키보드/저작권 기호
- [rule_72.rs]: placeholder marker
- [rule_73.rs]: 밑줄 빈칸
- [rule_74.rs]: 디지털 표기 기호

모든 규칙은 보통 다음 패턴을 가진다.

```
pub static META: RuleMeta = RuleMeta {
    section: "...",
    subsection: ...,
    name: "...",
    standard_ref: "...",
    description: "...",
};

pub struct RuleXX;

impl BrailleRule for RuleXX {
    fn meta(&self) -> &'static RuleMeta { &META }
    fn phase(&self) -> Phase { ... }
    fn priority(&self) -> u16 { ... }
    fn matches(&self, ctx: &RuleContext) -> bool { ... }
    fn apply(&self, ctx: &mut RuleContext) -> Result<RuleResult, String> { ... }
}
```

이 구조는 좋다. 규정 조항과 코드 위치가 직접 연결되기 때문이다. 새 규칙을 추가할 때도 이 틀을 따라야 한다.

10. 영어와 로마자 처리

영어 처리의 핵심은 다음 파일이다.

- [english.rs]: 영어 알파벳 기본 점자 셀 매핑
- [rule_en.rs]: 영어 약어/shortform 일부
- [english_logic.rs]: 영어 구간 유지/종료/문장부호 판단
- [rules/korean/rule_28.rs]: 영어 글자 인코딩 규칙

- [rules/korean/rule_29.rs]: 로마자 표지 처리
- [rules/korean/rule_english_symbol.rs]: 영어 문맥의 문장부호 처리

한글 문장 안의 영어는 일반 영어만 있는 문장과 다르게 처리된다. 예를 들어 (ATM 기기)처럼 한글과 섞이면 로마자 표지와 종료표가 들어간다. 반면 영어만 있는 입력은 불필요한 로마자 구간 표지가 생기지 않도록 처리해야 한다.

[english_indicator]라는 이름이 이 문맥 판단을 담당한다. 입력 전체에 한글 단어가 하나라도 있으면 영어 표지를 적극적으로 사용한다.

11. 숫자와 기호 처리

숫자 기본 매핑은 [number.rs]에 있다. 숫자가 처음 나올 때는 [rule_40.rs]가 수표를 붙인다.

기호는 크게 두 종류다.

- 일반 기호: [symbol_shortcut.rs]
- 수학 기호: [math_symbol_shortcut.rs]

문자 분류 단계에서 [is_symbol_char], [is_math_symbol_char]가 먼저 어떤 테이블로 보낼지 결정한다. 이 순서가 잘못되면 같은 문자도 일반 문장부호로 처리될지 수학 기호로 처리될지 달라진다.

12. 수학 점자 엔진

수학 점자는 [rules/math/] 아래에 별도 체계로 있다.

```
rules/math/
├── parser.rs
├── encoder.rs
├── math_token_rule.rs
├── function.rs
└── rule_1.rs ... rule_66.rs
```

12.1 MathToken

[parser.rs]는 수식을 [MathToken]으로 바꾼다.

```
pub enum MathToken {
    Variable(char),
    UpperVariable(char),
    Number(String),
    DecimalPoint,
    DigitSeparator,
    Operator(char),
    FunctionName(String),
    OpenParen(BracketKind),
    CloseParen(BracketKind),
    Superscript(Vec<MathToken>),
    Subscript(Vec<MathToken>),
    Space,
    MathSymbol(char),
    Prime,
```

```
Raw(char),
}
```

수학은 일반 텍스트보다 구조가 중요하다. 위첨자, 아래첨자, 괄호, 함수명, 분수, 근호, 적분은 문자 하나씩 매핑하면 처리하기 쉽다. 그래서 별도 토큰 파서가 있는 것이 맞다.

12.2 MathTokenEngine

{math_token_rule.rs}는 수학 토큰 규칙 인터페이스다. {MathTokenEngine}은 priority 순으로 규칙을 실행하고, 첫 번째로 매칭되는 규칙이 토큰을 소비한다.

{MathEncodeState}는 현재 상태로 {prev_was_number}, {logic_context}를 들고 있다. 수학에서는 이전 토큰이 숫자인지, 논리 기호 문맥인지에 따라 spacing이나 표지가 달라질 수 있다.

12.3 수학식 감지

일반 입력 안에서 어떤 단어를 수학식으로 볼지는 {token_rules/math_expression.rs}가 판단한다. 여기에는 다음과 같은 감지 조건이 들어 있다.

- 위첨자/아래첨자 문자가 있는가
- 결합 수학 기호가 있는가
- 함수명으로 시작하는가
- 근호로 시작하는가
- 절댓값 형태인가
- 연산자/관계기호/괄호가 있는가
- 한글 접미사와 붙어 있는 혼합 수식인가

이 파일은 크고 조건도 많다. 수학 관련 회귀가 생기기 쉬운 곳이다.

12.4 LaTeX 처리

LaTeX 처리는 {token_rules/latex_math.rs}, {token_rules/latex_fraction.rs}, {fraction.rs}가 담당한다.

지원하는 대표 입력은 다음 계열이다.

- $\{\frac{3}{4}\}$
- $\{\sqrt{x}\}$
- $\{x^2\}$
- $\{x_n\}$
- $\{\neq\}$, $\{\geq\}$, $\{\leq\}$
- $\{\int f(x)dx\}$
- $\{\cup\}$, $\{\cap\}$, $\{\subset\}$
- $\{\forall\}$, $\{\exists\}$

테스트케이스 규칙상 LaTeX 입력은 기존 PDF 예제의 LaTeX 버전이어야 한다. 새로운 수식을 임의로 만들면 안 된다.

13. Formatting API

{FormattingSpan}과 {FormattingKind}는 입력 문자열의 byte range에 강조/굵게/사용자 정의 글자체를 지정하는 API다.

```
pub enum FormattingKind {  
    Emphasis,  
    Bold,  
    Custom1,  
    Custom2,  
}
```

{encode_with_formatting}은 먼저 일반 토큰화를 하고, 지정된 byte range 경계에 {Token::PreEncoded} marker를 삽입한다. 중요한 점은 range가 UTF-8 char boundary에 맞아야 한다는 것이다. 한글 문자열에서는 byte offset과 char index가 다르므로 호출자가 조심해야 한다.

14. Node/WASM 바인딩

{packages/node}는 {wasm-bindgen} 기반이다.

```
#{wasm_bindgen(js_name = "encode")}  
pub fn encode(text: &str) -> Result<Vec<u8>, String>  
  
#{wasm_bindgen(js_name = "translateToUnicode")}  
pub fn translate_to_unicode(text: &str) -> Result<String, String>  
  
#{wasm_bindgen(js_name = "translateToBrailleFont")}  
pub fn translate_to_braille_font(text: &str) -> Result<String, String>
```

{package.json}의 {main}, {module}, {exports}는 모두 {pkg/index.js}를 가리킨다. 따라서 배포 전에는 반드시 {wasm-pack build --target bundler --out-dir ./pkg --out-name index}를 실행해야 한다.

웹사이트와 모바일 앱은 이 Node/WASM 패키지를 직접 import한다.

15. Python 바인딩

{packages/python}은 {pyo3}와 {maturin}을 사용한다.

노출 함수는 다음과 같다.

- {encode(text) -> list(int)}
- {translate_to_unicode(text) -> str}
- {translate_to_braille_font(text) -> str}
- {cli() }

에러는 Rust (String) 에러를 Python (ValueError)로 바꾼다. CLI는 Rust의 (run_cli)를 그대로 호출한다.

16. .NET 바인딩

.NET 바인딩은 두 층이다.

1. Rust (packages/dotnet/src/lib.rs)가 C ABI 함수를 제공한다.
2. C# (packages/dotnet/BraillifyNet)가 (DllImport)/(LibraryImport)로 호출한다.

Rust 쪽 exported 함수는 다음과 같다.

- (braillify_encode)
- (braillify_encode_to_unicode)
- (braillify_encode_to_braille_font)
- (braillify_get_last_error)
- (braillify_free_string)
- (braillify_free_bytes)

메모리 소유권이 중요하다. Rust가 할당한 문자열과 바이트 배열은 C#이 사용 후 반드시 (braillify_free_string) 또는 (braillify_free_bytes)로 해제해야 한다. C# 래퍼는 (finally)에서 이 해제를 수행한다.

(NativeLibraryLoader)는 NuGet 패키지의 (runtimes/{rid}/native/) 구조에서 플랫폼별 native library를 찾아 로드한다. 이 구조는 .NET 패키지 배포에 적합하다.

17. CLI

CLI는 (libs/braillify/src/cli.rs)와 (main.rs)에 있다.

동작 방식은 단순하다.

- 인자가 있으면 그 문자열을 한 번 점역해서 stdout에 출력한다.
- 인자가 없고 stdin pipe가 있으면 stdin 내용을 읽어 점역한다.
- 인자도 pipe도 없으면 REPL을 실행한다.

(clap), (anyhow), (rustyline)은 (cli) feature에 묶여 있다. 라이브러리만 쓰는 빌드에서는 CLI 의존성을 뺄 수 있다.

18. Landing 웹사이트

(apps/landing)은 Next.js 앱이다.

주요 화면은 다음과 같다.

- [/]: 랜딩 홈과 실시간 점역 체험
- [/docs/overview]: 개요 문서
- [/docs/installation]: 설치 문서
- [/docs/api]: API 문서
- [/docs/contributing]: 기여 문서
- [/team]: 팀 페이지
- [/test-case]: 테스트케이스 대시보드

[Trans.tsx]는 클라이언트 컴포넌트로 [import('braillify')]를 동적으로 호출한다. 이 방식은 WASM 로딩을 브라우저에서 처리하기 위해 필요하다.

[test-case/page.tsx]는 서버 컴포넌트에서 [rule_map.json], [test_status.json]을 읽는다. [test_status.json]은 Rust 테스트 [test_by_testcase]가 생성하는 파일이다. 즉 landing의 테스트 대시보드는 빌드 전에 테스트 상태 파일이 있어야 제대로 렌더링된다.

19. Mobile/Tauri 앱

[apps/mobile]은 현재 로컬 작업물로 보인다. 구조는 다음과 같다.

```
apps/mobile/
├── src/
│   ├── App.tsx
│   ├── pages/
│   │   ├── TranslatorPage.tsx
│   │   ├── EditorPage.tsx
│   │   └── HistoryPage.tsx
│   ├── components/
│   └── lib/
│       ├── translate.ts
│       ├── braille.ts
│       ├── clipboard.ts
│       └── history.ts
└── src-tauri/
```

[translate.ts]는 [braillify] WASM의 [translateToUnicode]를 호출한다. 수학 모드에서는 [\$...\$] 형식과 종괄호 짝만 간단히 검증한다. 실제 수학 점역은 여전히 Rust 코어가 한다.

[braille.ts]는 6점 점자 셀 편집을 위한 helper다.

- dot 번호를 bit mask로 변환
- mask를 Unicode Braille 문자로 변환
- 점자 문자열을 mask 배열로 파싱
- 음각 출력을 위해 dot1/dot4, dot2/dot5, dot3/dot6을 swap

이 앱은 라이브러리 소비 예제로도 가치가 있다. 다만 현재 비추적 상태이므로 커밋/리뷰 전에 의도한 추가인지 확인해야 한다.

20. 테스트케이스

[test_cases]는 사실상 제품 명세다.

현재 로컬 기준:

- JSON 파일: 156개
- 전체 엔트리: 2064개
- 한국어 엔트리: 1487개
- 수학 엔트리: 577개

파일 이름 규칙:

- [test_cases/korean/rule_{N}.json]
- [test_cases/korean/rule_{N}_b1.json]
- [test_cases/math/math_{N}.json]

각 엔트리는 다음 구조다.

```
{
  "input": "입력 텍스트",
  "note": "설명",
  "internal": "점자 내부표기",
  "expected": "브라유셀 인덱스 연결 문자열",
  "unicode": "점자 유니코드 문자열"
}
```

[testcase-integrity.test.ts]는 [internal]에서 [expected]와 [unicode]가 정확히 계산되는지 검증한다. 이때 [braillove-case-collector/converter.py]와 같은 패턴을 사용한다.

중요한 개발 원칙:

- PDF에 없는 예제를 임의로 만들면 안 된다.
- PDF 순서대로 넣어야 한다.
- 기호는 단독 엔트리를 먼저 넣어야 한다.
- [note]는 필요한 때만 써야 한다.
- LaTeX는 기존 예제의 다른 입력 형식으로만 추가해야 한다.

21. 테스트 실행 구조

루트 [bun run test]는 매우 무겁다.

대략 다음을 실행한다.

1. [cargo tarpaulin]

2. `{bun test --coverage}`
3. `{cd py-test && uv run pytest __init__.py}`

Rust 내부에는 `{test_by_testcase}`라는 큰 테스트가 있다. 이 테스트는 모든 `test_cases`를 돌며 실제 엔진 출력과 정답을 비교하고 `{test_status.json}`을 생성한다. landing의 테스트 대시보드는 이 결과를 사용한다.

Bun 테스트는 Node/WASM 바인딩의 기본 동작과 `test_cases integrity`를 검증한다.

Python 테스트는 설치된 Python extension module을 통해 `{translate_to_unicode}`가 동작하는지 확인한다.

22. CI/CD

`{.github/workflows/publish.yml}`은 main push와 PR target에서 동작한다. 주요 job은 다음과 같다.

- Rust/Node/Python 통합 테스트
- .NET native library build와 .NET 테스트
- landing deploy

`{.github/workflows/publish-pypi.yml}`은 release 또는 수동 실행으로 Python wheel과 cargo publish 관련 작업을 수행한다.

주의할 점은 `{pull_request_target}`과 `{permissions: write-all}` 조합이다. 오픈소스 저장소에서는 보안상 조심해야 하는 설정이다. 외부 PR에서 임의 코드 실행과 secret 접근 가능성에 특히 주의해야 한다.

23. 코드 품질 리뷰

좋은 점

1. 코어가 Rust 하나로 집중되어 있다.

Node/Python/.NET이 각각 자체 구현을 하지 않고 Rust 코어만 호출한다. 이 구조는 유지보수에 매우 좋다.

2. 규정 조항과 코드 파일이 연결된다.

`{rule_11.rs}`, `{rule_12.rs}`처럼 조항별 파일이 있어 PDF와 코드를 대조하기 쉽다.

3. `{RuleMeta}`가 있다.

규칙이 어느 조항의 어떤 설명인지 코드 안에서 추적할 수 있다.

4. 토큰 단계와 문자 단계를 나눴다.

수식, LaTeX, 대문자 passage, 로마 숫자, 분수는 문자 하나씩 처리하면 어려운데, IR 단계가 있어서 확장성이 있다.

5. 테스트케이스가 별도 데이터로 관리된다.

이 프로젝트에서는 테스트 데이터가 곧 표준 구현의 근거다.

아쉬운 점

1. `{lib.rs}`에 테스트와 회귀 케이스가 너무 많이 몰려 있다.
공개 API 파일이 테스트 파일 역할까지 크게 맡고 있어 읽기가 무거워진다.
2. `{emit_word}`의 상태 전이가 복잡하다.
영어 모드, 숫자 모드, 괄호, continuation, 대문자 상태가 한 loop에 모여 있다. 회귀 위험이 크다.
3. `{math_expression.rs}`, `{latex_math.rs}`가 너무 크다.
수학 감지와 변환 조건이 늘어나면서 조건 분기가 많은 파일이 되었다. 기능 추가 시 부작용을 찾기 어렵다.
4. `{encode_to_braille_font}`의 이름과 동작이 애매하다.
현재는 Unicode Braille 출력과 동일하다. 실제 폰트용 출력이 아니라면 문서에서 명확히 설명해야 한다.
5. 오류 타입이 전부 `{String}`이다.
라이브러리 안정화 단계에서는 enum 기반 error type이 있으면 바인딩별 에러 처리와 디버깅이 좋아진다.

24. 개발자가 기능 추가할 때의 안전한 순서

새 점자 규칙이나 테스트케이스를 추가할 때는 다음 순서를 추천한다.

1. `{docs/2024 개정 한국 점자 규정.pdf}`에서 근거를 확인한다.
2. 해당 조항의 `{test_cases}` JSON에 예제를 추가한다.
3. `{braillove-case-collector/converter.py}` 규칙대로 `{internal}`, `{expected}`, `{unicode}`를 맞춘다.
4. 기존 규칙 파일이 있는지 확인한다.
5. 없으면 `{rules/korean/rule_N.rs}` 또는 `{rules/math/rule_N.rs}`를 만든다.
6. `{RuleMeta}`를 정확히 작성한다.
7. `{Encoder::new}` 또는 `math encoder` 등록 위치에 규칙을 넣는다.
8. 가장 작은 단위 테스트를 추가한다.
9. `{bun test test_cases/}`로 integrity를 확인한다.
10. 관련 Rust 테스트 또는 `{cargo test test_by_testcase}`를 돌린다.

가장 피해야 할 방식은 "출력만 맞추기 위해 `{solvable_case_override}` 같은 예외 테이블에 계속 추가하는 것"이다.
단기적으로는 테스트가 통과하지만, 표준 구현으로서는 유지보수가 어려워진다.

25. 처음 합류한 개발자를 위한 읽기 로드맵

1일차: 전체 구조

- `{README.md}`

- {package.json}
- {Cargo.toml}
- {libs/brailify/Cargo.toml}
- {libs/brailify/src/lib.rs}

목표: 외부 API와 빌드 구조 이해.

2일차: 기본 한글 접역

- {char_struct.rs}
- {korean_char.rs}
- {jauem/choseong.rs}
- {jauem/jongseong.rs}
- {moeum/jungsong.rs}
- {char_shortcut.rs}
- {word_shortcut.rs}

목표: {안녕하세요}가 어떻게 점자 셀로 바뀌는지 손으로 따라가기.

3일차: 규칙 엔진

- {rules/token.rs}
- {rules/token_engine.rs}
- {rules/token_rule.rs}
- {rules/traits.rs}
- {rules/engine.rs}
- {rules/emit.rs}

목표: 토큰 규칙과 문자 규칙의 차이 이해.

4일차: 영어/숫자/기호

- {english.rs}
- {english_logic.rs}
- {number.rs}
- {symbol_shortcut.rs}
- {rules/korean/rule_28.rs}
- {rules/korean/rule_29.rs}
- {rules/korean/rule_40.rs}

- [rules/korean/rule_49.rs]

목표: 한글 문장 안의 영어, 숫자, 문장부호가 왜 복잡한지 이해.

5일차: 수학

- [rules/math/parser.rs]
- [rules/math/encoder.rs]
- [rules/math/math_token_rule.rs]
- [rules/token_rules/math_expression.rs]
- [rules/token_rules/latex_math.rs]
- [fraction.rs]

목표: $\{\frac{3}{4}\}$, $\{x^2\}$, $\{1/2\}$ 가 어느 경로로 처리되는지 이해.

6일차: 바인딩과 앱

- [packages/node/src/lib.rs]
- [packages/python/src/lib.rs]
- [packages/dotnet/src/lib.rs]
- [apps/landing/src/app/Trans.tsx]
- [apps/mobile/src/lib/translate.ts]

목표: Rust 코어가 각 플랫폼으로 어떻게 노출되는지 이해.

26. 결론

Braillify는 "한글 점자 변환 라이브러리"처럼 보이지만 실제로는 다음 네 가지가 결합된 프로젝트다.

1. 2024 개정 한국 점자 규정 구현체
2. 수학/중세국어/영어/기호까지 포함하는 텍스트 처리 엔진
3. Rust 코어를 여러 언어로 배포하는 크로스플랫폼 패키지
4. 테스트케이스와 웹 대시보드를 통한 표준 구현 검증 시스템

숙련된 개발자 관점에서 가장 중요한 조언은 이것이다.

기능을 추가할 때 "코드를 먼저" 보지 말고 "PDF 조항 -> 테스트케이스 -> 기존 규칙 위치 -> 엔진 등록 순서" 순서로 접근해야 한다. 이 프로젝트에서 정답은 코드 안에만 있지 않고, PDF와 test_cases 안에 같이 있다.